ECOLE MOHAMMADIA D'INGÉNIEURS

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

École Mohammadia d'Ingénieurs - Rabat

Développement Web avec Python
Système d'authentification Django

Atelier 3 : Authentification avec Django

Élève :
Nada Chahbane

Professeur :
Mr. A. Bousselham

Année Universitaire 2025-2026

Résumé

Ce rapport présente les travaux réalisés dans le cadre de l'**Atelier 3** du module *Développement Web avec Python*. L'objectif est d'ajouter un système d'**authentification complet** dans le projet e-commerce Django existant, en créant une nouvelle application `accounts` gérant l'inscription, la connexion, la déconnexion et la page profil utilisateur.

Nous avons appris à utiliser le système d'authentification intégré de Django, à créer des formulaires personnalisés, à protéger des vues avec le décorateur `@login_required`, et à réutiliser un template de base existant.

Mots-clés : Django, Authentification, Session, CSRF, login_required, UserCreationForm, accounts.

Objectifs de l'atelier

- Ajouter une application `accounts` dans un projet Django existant.
- Utiliser le système d'authentification intégré de Django.
- Créer une page d'**inscription** avec formulaire personnalisé.
- Créer une page de **connexion**.
- Gérer la **déconnexion**.
- Créer une page **profil protégée** accessible uniquement aux utilisateurs connectés.
- Réutiliser le template `base.html` existant.
- Intégrer l'authentification dans le projet e-commerce.

Table des matières

Résumé **1**

Objectifs de l'atelier **2**

1 Architecture du Projet **4**
1.1 Architecture avant l'atelier 4
1.2 Architecture après l'atelier 4
1.3 Schéma de l'architecture d'authentification 6

2 Mise en Place de l'Application Accounts **7**
2.1 Création de l'application 7
2.2 Enregistrement dans settings.py 7
2.3 Configuration des redirections 7
2.4 Configuration des templates 7

3 Formulaire, Vues et URLs **9**
3.1 Formulaire d'inscription (forms.py) 9
3.2 Vues (views.py) 9
3.3 URLs de accounts (accounts/urls.py) 10
3.4 URLs principales (ecommerce/urls.py) 10

4 Templates HTML **12**
4.1 Modification de base.html 12
4.2 Page de connexion (login.html) 12
4.3 Page d'inscription (signup.html) 13
4.4 Page profil (profile.html) 13

5 Tests et Résultats **15**
5.1 Liste des produits (utilisateur connecté) 15
5.2 Page de connexion 15
5.3 Page d'inscription 16
5.4 Page profil (connecté) 16
5.5 Redirection automatique (non connecté) 17
5.6 Interface d'administration Django 17
5.7 Tableau récapitulatif des tests 18

6 Notions Importantes **19**
6.1 Authentification, Session et Sécurité 19
6.2 Le décorateur @login_required 19
6.3 Le jeton {% csrf_token %} 19
6.4 Schéma du flux d'authentification 20

7 Difficultés Rencontrées **21**

Conclusion **22**

Chapitre 1

Architecture du Projet

1.1 Architecture avant l'atelier

Avant cet atelier, le projet Django e-commerce comportait une seule application `products` avec la structure suivante :

Structure initiale du projet

```
ecommerce/
  ecommerce/
    settings.py  urls.py  asgi.py  wsgi.py
  images/
  products/
    migrations/
    templates/
      base.html
      product_detail.html
      product_list.html
    admin.py  apps.py  models.py  urls.py  views.py
  db.sqlite3
  manage.py
  myenv/
```

1.2 Architecture après l'atelier

Après l'ajout de l'application `accounts`, la structure devient :

Structure finale du projet

```
ecommerce/
  ecommerce/
    settings.py  urls.py  asgi.py  wsgi.py
  images/
  products/
    templates/
      base.html
      product_detail.html
      product_list.html
    urls.py  views.py  ...
  accounts/                    <-- NOUVEAU
    templates/
```

```
      registration/
        login.html
        signup.html
        profile.html
    forms.py  urls.py  views.py  ...
  db.sqlite3
  manage.py
  myenv/
```

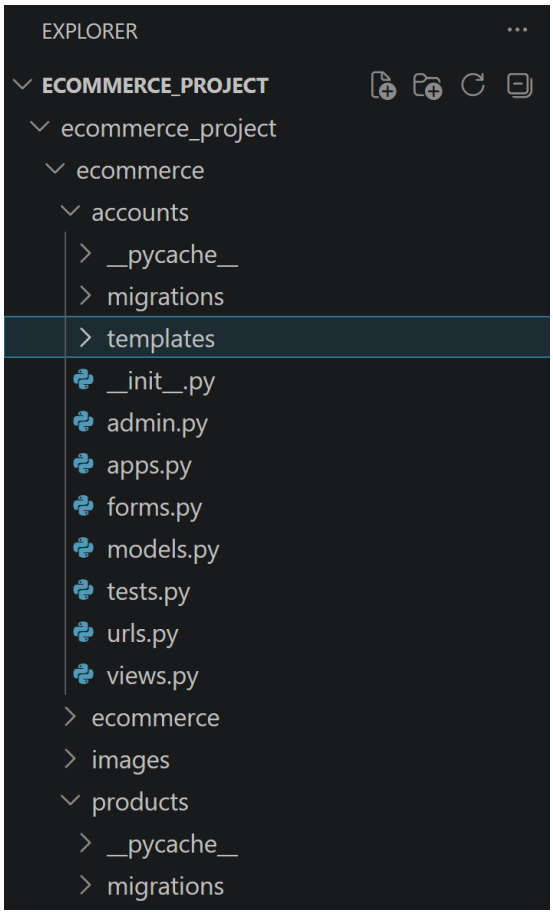

FIGURE 1.1 – Architecture du projet dans VS Code

1.3 Schéma de l'architecture d'authentification

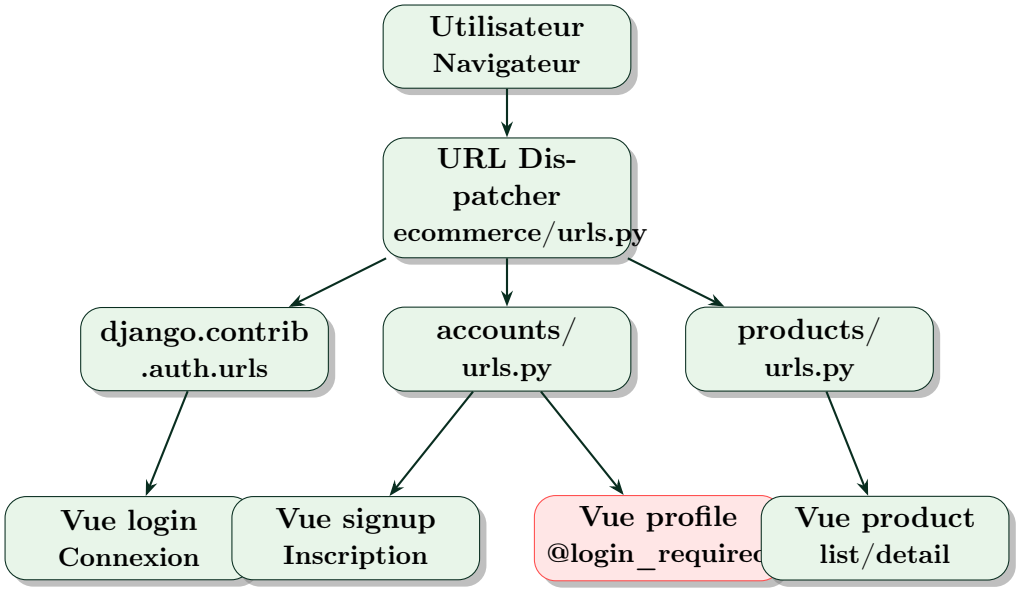

Figure : Architecture des URLs et vues du projet après l'atelier

Chapitre 2

Mise en Place de l'Application Accounts

2.1 Création de l'application

```
python manage.py startapp accounts
```

Listing 2.1 – Création de l'application accounts

2.2 Enregistrement dans settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'products',
    'accounts',    # Nouvelle application
]
```

Listing 2.2 – settings.py — INSTALLED_APPS

2.3 Configuration des redirections

```
LOGIN_REDIRECT_URL  = "profile"        # Apres connexion reussie
LOGOUT_REDIRECT_URL = "product_list"   # Apres deconnexion
LOGIN_URL           = "login"          # Page de connexion
```

Listing 2.3 – settings.py — Redirections d'authentification

Rôle des variables de redirection

- `LOGIN_REDIRECT_URL` : redirige vers le profil après connexion.
- `LOGOUT_REDIRECT_URL` : redirige vers les produits après déconnexion.
- `LOGIN_URL` : redirige vers la connexion si page protégée accédée sans authentification.

2.4 Configuration des templates

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.
           DjangoTemplates',
        'DIRS': [
            BASE_DIR / 'accounts' / 'templates',
        ],
        'APP_DIRS': True,
        ...
    },
]
```

Listing 2.4 – settings.py — DIRS des templates

Chapitre 3

Formulaire, Vues et URLs

3.1 Formulaire d'inscription (forms.py)

```python
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True, label="Adresse email")

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]
```

Listing 3.1 – accounts/forms.py

UserCreationForm

`UserCreationForm` est un formulaire fourni par Django qui gère automatiquement la validation du mot de passe. Il est personnalisé ici pour inclure le champ `email`.

3.2 Vues (views.py)

```python
from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .forms import RegisterForm

def signup(request):
    if request.method == "POST":
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, "Compte cree avec succes.")
            return redirect("profile")
    else:
        form = RegisterForm()
    return render(request, "registration/signup.html", {"form": form})
```

```python

@login_required
def profile(request):
    return render(request, "registration/profile.html")
```

Listing 3.2 – accounts/views.py

Décorateur @login_required

Le décorateur `@login_required` protège la vue `profile`. Si un utilisateur non connecté tente d'y accéder, Django le redirige automatiquement vers la page de connexion avec le paramètre `?next=/accounts/profile/`.

3.3 URLs de accounts (accounts/urls.py)

```python
from django.urls import path
from . import views

urlpatterns = [
    path("signup/", views.signup, name="signup"),
    path("profile/", views.profile, name="profile"),
]
```

Listing 3.3 – accounts/urls.py

3.4 URLs principales (ecommerce/urls.py)

```python
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static
from django.views.generic import RedirectView

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', RedirectView.as_view(
             pattern_name='product_list', permanent=False)),
    path('products/', include("products.urls")),
    path('accounts/', include("accounts.urls")),
    path('accounts/', include("django.contrib.auth.urls")),
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Listing 3.4 – ecommerce/urls.py

URLs d'authentification Django

La ligne `include("django.contrib.auth.urls")` active automatiquement : `/accounts/login/`, `/accounts/logout/`, `/accounts/password_change/`, `/accounts/password_reset/`

Chapitre 4

Templates HTML

4.1 Modification de base.html

```
{% if user.is_authenticated %}
    <a href="{% url 'profile' %}">Mon profil</a>
    <span>Bonjour, {{ user.username }}</span>
    <form method="post" action="{% url 'logout' %}">
        {% csrf_token %}
        <button type="submit">Deconnexion</button>
    </form>
{% else %}
    <a href="{% url 'login' %}">Connexion</a>
    <a href="{% url 'signup' %}">Creer un compte</a>
{% endif %}
```

Listing 4.1 – base.html — Navbar avec authentification

Jeton CSRF

La balise `{% csrf_token %}` est obligatoire dans tout formulaire POST. Elle protège l'application contre les attaques **Cross-Site Request Forgery**.

4.2 Page de connexion (login.html)

```
{% extends 'base.html' %}
{% block title %}Connexion{% endblock %}
{% block content %}
<div class="row justify-content-center mt-5">
  <div class="col-md-5">
    <h2>Connexion</h2>
    <form method="post">
      {% csrf_token %}
      {{ form.as_p }}
      <button type="submit" class="btn btn-primary">
        Se connecter
      </button>
    </form>
    <p class="mt-3">
      Pas encore de compte ?
      <a href="{% url 'signup' %}">Creer un compte</a>
    </p>
  </div>
```

```
</div>
{% endblock %}
```

Listing 4.2 – accounts/templates/registration/login.html

4.3 Page d'inscription (signup.html)

```
{% extends 'base.html' %}
{% block title %}Creer un compte{% endblock %}
{% block content %}
<div class="row justify-content-center mt-5">
  <div class="col-md-6">
    <h2>Creation d'un compte</h2>
    <form method="post">
      {% csrf_token %}
      {{ form.as_p }}
      <button type="submit" class="btn btn-success">
        Creer mon compte
      </button>
    </form>
    <p class="mt-3">
      Deja un compte ?
      <a href="{% url 'login' %}">Se connecter</a>
    </p>
  </div>
</div>
{% endblock %}
```

Listing 4.3 – accounts/templates/registration/signup.html

4.4 Page profil (profile.html)

```
{% extends 'base.html' %}
{% block title %}Mon profil{% endblock %}
{% block content %}
<div class="mt-5">
  <h2>Mon profil</h2>
  <p>Page protegee, accessible aux utilisateurs connectes.</p>
  <h4>Informations du compte</h4>
  <ul class="list-group">
    <li class="list-group-item">
      Nom : {{ user.username }}
    </li>
    <li class="list-group-item">Email : {{ user.email }}</li>
    <li class="list-group-item">Staff : {{ user.is_staff }}</li>
    <li class="list-group-item">
      Superutilisateur : {{ user.is_superuser }}
    </li>
```

```
  </ul>
</div>
{% endblock %}
```

Listing 4.4 – accounts/templates/registration/profile.html

Chapitre 5

Tests et Résultats

5.1 Liste des produits (utilisateur connecté)

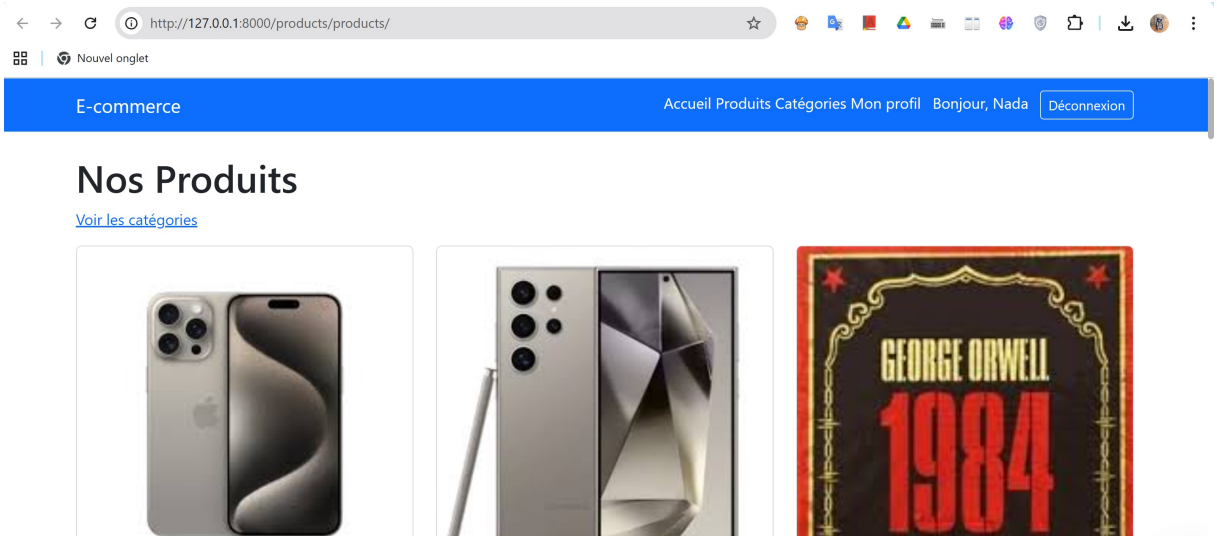

FIGURE 5.1 – Liste des produits avec utilisateur connecté

5.2 Page de connexion

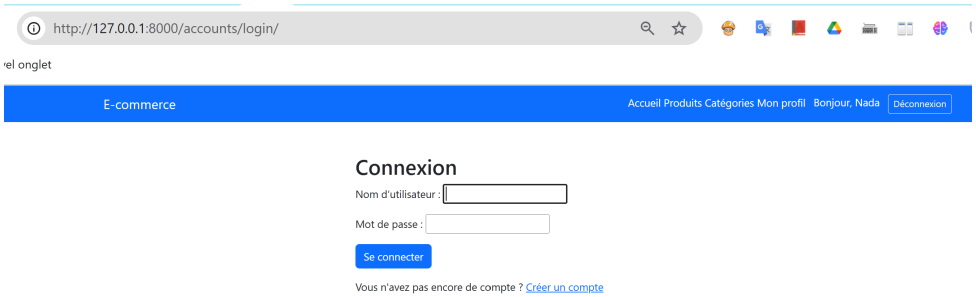

FIGURE 5.2 – Page de connexion

5.3 Page d'inscription

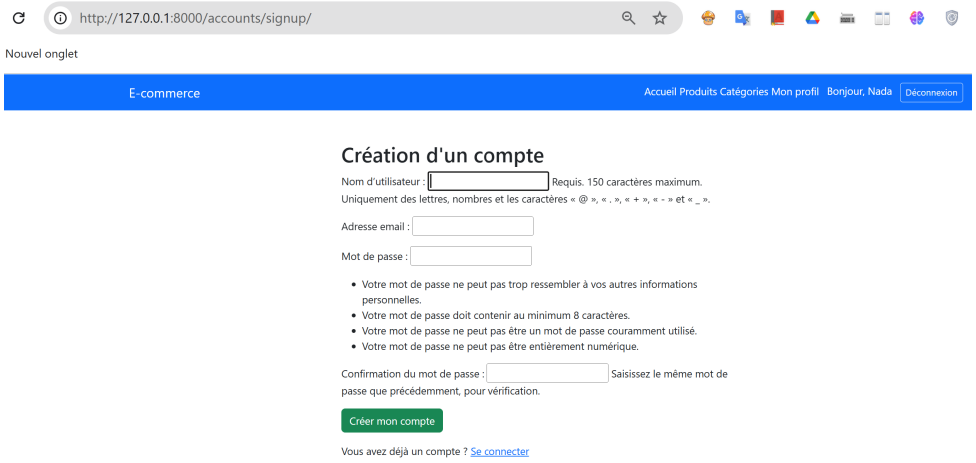

FIGURE 5.3 – Page d'inscription

5.4 Page profil (connecté)

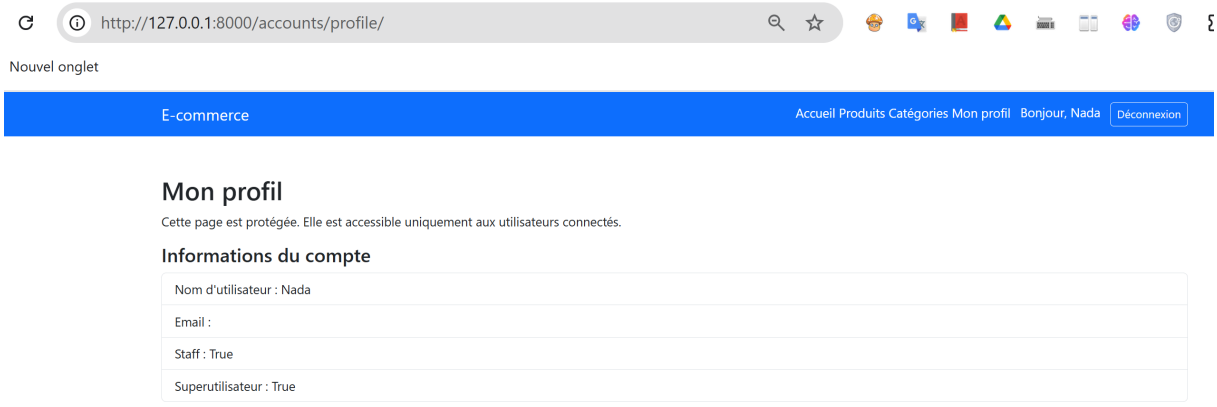

FIGURE 5.4 – Page profil après connexion

5.5 Redirection automatique (non connecté)

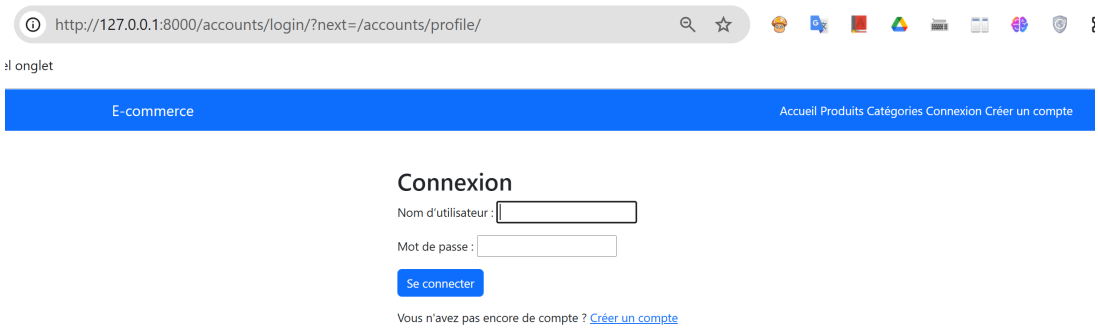

FIGURE 5.5 – Redirection automatique vers la connexion si non connecté

5.6 Interface d'administration Django

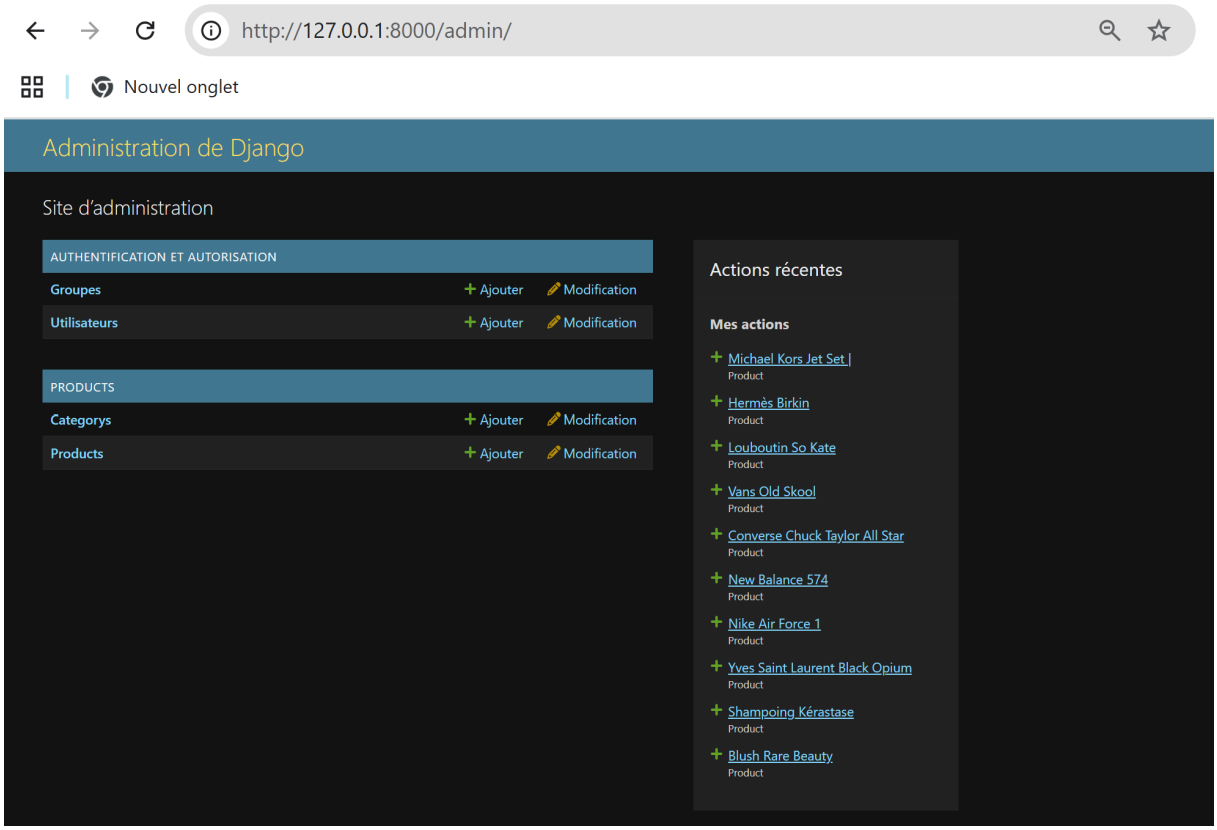

FIGURE 5.6 – Interface d'administration Django

5.7 Tableau récapitulatif des tests

URL	Description	Résultat
`/products/`	Liste des produits avec images	**OK**
`/accounts/signup/`	Création d'un nouveau compte	**OK**
`/accounts/login/`	Connexion avec identifiants	**OK**
`/accounts/profile/`	Profil (connecté)	**OK**
`/accounts/profile/`	Profil (non connecté)	**Redirigé**
`/accounts/logout/`	Déconnexion	**OK**
`/admin/`	Interface administration	**OK**

Chapitre 6

Notions Importantes

6.1 Authentification, Session et Sécurité

Notion	Explication
Authentification	Vérification de l'identité via nom d'utilisateur et mot de passe.
Session	Mécanisme qui maintient l'utilisateur connecté entre les pages.
Mot de passe haché	Django ne stocke jamais les mots de passe en clair. Il utilise PBKDF2 pour les sécuriser.
CSRF Token	Jeton anti-falsification dans chaque formulaire POST.

6.2 Le décorateur @login_required

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    return render(request, "registration/profile.html")
```

Listing 6.1 – Utilisation de @login_required

Comportement de @login_required

- Utilisateur **connecté** : la vue s'exécute normalement.
- Utilisateur **non connecté** : Django redirige vers `LOGIN_URL` avec le paramètre `?next=<url>` pour revenir après connexion.

6.3 Le jeton {% csrf_token %}

Rôle de csrf_token

La balise `{% csrf_token %}` génère un jeton unique et secret inséré dans le formulaire. Lors de la soumission, Django vérifie ce jeton pour s'assurer que la requête provient bien du formulaire légitime et non d'un site malveillant.

6.4 Schéma du flux d'authentification

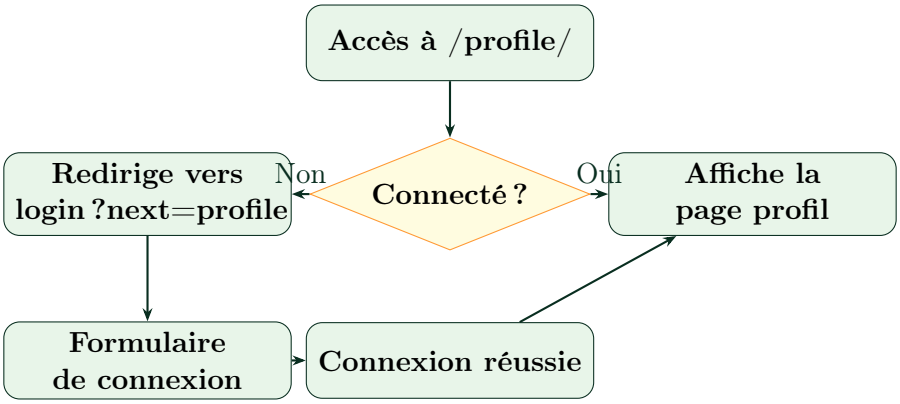

Figure : Flux d'authentification avec @login_required

Chapitre 7

Difficultés Rencontrées

— **Erreur** `TemplateDoesNotExist` **:** Django ne trouvait pas `registration/profile.html` car il cherchait uniquement dans `products/templates/`. Résolue en ajoutant `BASE_DIR / 'accounts' / 'templates'` dans `DIRS` de `TEMPLATES` dans `settings.py`.

— **Connexion PostgreSQL refusée :** Lors de la configuration, l'authentification échouait car aucun mot de passe n'avait été défini. Résolue en modifiant `pg_hba.conf` pour utiliser la méthode `trust` et en redémarrant le service PostgreSQL en mode administrateur.

— **Mauvaise arborescence du projet :** Plusieurs niveaux de dossiers imbriqués causaient des erreurs *No such file or directory* pour `manage.py`. Résolue en naviguant au bon niveau avec `cd`.

— **Template** `base.html` **au lieu de** `layout.html` **:** L'atelier référençait `layout.html` mais le projet utilisait `base.html`. Tous les `extends` ont été adaptés.

Conclusion

Cet atelier nous a permis d'implémenter un système d'authentification complet dans notre application e-commerce Django. Nous avons acquis les compétences suivantes :

- Créer et intégrer une nouvelle application Django (`accounts`).
- Utiliser `UserCreationForm` pour un formulaire d'inscription personnalisé.
- Protéger des vues avec le décorateur `@login_required`.
- Comprendre le mécanisme de **session** et de **hachage de mot de passe**.
- Utiliser les URLs d'authentification intégrées de Django.
- Afficher du contenu conditionnel dans les templates selon l'état de connexion.

Point clé

Ces compétences constituent une base solide pour les fonctionnalités e-commerce avancées : panier protégé, historique des commandes, gestion du profil utilisateur.